10/15: Lecture 15. Space Complexity

1 Resources So Far

We've already studied the following resources:

- Time Complexity
- Nondeterminism
- Interaction

The next resource we will explore is **space complexity**. Some questions we will consider are: if we have an extremely fast algorithm that solves a problem, are we able to reduce its memory to polynomial space? How do space and time complexity compare? Finally, how does nondeterminism interact with space?

2 Space Complexity

2.1 Definitions

Deterministic space complexity:

$$\text{SPACE}[S(n)] = \{L : L \text{ is solved by a NTM using } c \cdot S(n) \text{ total work space }\}$$

Nondeterministic space complexity:

$$\text{NSPACE}[S(n)] = \{L : L \text{ is solved by a NTM using } c \cdot S(n) \text{ total work space on every branch}\}$$

Given the input encoding on a special, read-only "input" tape, we count the **total work space** as the number of cells the TM ever touches besides the input tape.

- We define the total work space this way because we would like to study **sublinear space complexity** (i.e. the auxiliary memory usage is strictly less than linear in terms of the input size)
- Note: It doesn't make sense for us to study sublinear *time* complexity, because we would, at the very least, need to read the entire input, which takes linear time.

2.2 Space Complexity Classes

Poly-space:

$$\text{PSPACE} = \bigcup_{c \in \mathbb{N}} \text{SPACE}[n^c]$$

Nondeterministic poly-space:

$$\text{NPSPACE} = \bigcup_{c \in \mathbb{N}} \text{NSPACE}[n^c]$$

Logspace:

$$\text{L} = \bigcup_{c \in \mathbb{N}} \text{SPACE}[\log n]$$

Nondeterministic logspace:

$$\text{NL} = \bigcup_{c \in \mathbb{N}} \text{NSPACE}[\log n]$$

- Clearly, $\text{L} \subseteq \text{PSPACE}$.

3 Comparison of Time and Space Complexity

Fact.

$$\text{TIME}[S(n)] \subseteq \text{NTIME}[S(n)] \subseteq \text{SPACE}[S(n)] \subseteq \text{NSPACE}[S(n)]$$

- Observe that any algorithm that finishes within (nondeterministic) poly-time can only visit/write to a polynomial amount of memory.

3.1 NP, Co-NP, and PSPACE

Facts.

$$\text{NP} \subseteq \text{PSPACE}$$

$$\text{CoNP} \subseteq \text{PSPACE}$$

Example. 3-SAT

- **Poly-Space Solution.** For some language $L \in \text{NP}$, L has a poly-time reduction to 3-SAT, so it is also a poly-time space reduction. After, for each possible assignment $a \in \{0,1\}^n$ (reusing the space each time), we check if $\Phi(a) = 1$.
- **Exponential Alternative: Exhaustive Search.** First, write every single possible assignment $a \in \{0,1\}^n$ onto the tape. Go through each a on the tape and check if $\Phi(a) = 1$.

3.2 Bounding Deterministic Space with Time

Claim.

$$\text{SPACE}[S(n)] \subseteq \text{TIME}[2^{O(S(n))}]$$

Proof. (By counting argument)

- Goal: We want to count the total number of possible configurations.
- Since there are $2^{O(S(n))}$ unique configurations, we can encode each configuration in $O(S(n))$ bits.

Suppose that some $L \in \text{SPACE}[S(n)]$ with a machine M. On an input x, simulate running M while maintaining a counter $c \leftarrow 0$ (initialized to zero). During this simulation:

- If M accepts, ACC.
- If M rejects, REJ.
- After every step, increment $c \leftarrow c + 1$.
- If $c = 2^{100 \cdot S(n)}$, REJ (By the Pigeonhole Principle, if we run for more steps than the number of possible configurations, we know we are in an infinite loop and should stop, since $x \notin L$).

3.3 Bounding Nondeterministic Space with Time

Claim.

$$\text{NSPACE}[S(n)] \subseteq \text{TIME}[2^{O(S(n))}]$$

Proof. (By configuration graph)
Suppose that $L \in \text{NSPACE[S(n)]}$ with a NTM M, where $S(n) \geq \log n$.

Construction. We construct a *configuration graph* $G = (V, E)$ representing all configurations of M on input x:

- Vertices: Each vertex represents a possible configuration of M (i.e. its current state Q_{cur}, tape contents, and tape head positions). Since M uses $O(S(n))$ space, there are $2^{O(S(n))}$ possible configurations (assuming $S(n) \geq \log n$).
- Edges: $(C, C') \in E$ iff C' can be obtained from C by one legal transition of M, according to its transition function δ.

- Outdegree: A DTM has at most one outgoing edge per configuration. Notice that a NTM has $O(1)$ outgoing edges, since its transition function allows a constant number of choices. Without loss of generality, we can assume at most two choices per step.
- Accepting Configuration: Without loss of generality, assume there is a single accepting configuration C_{acc}.

Let C_{start} be the initial configuration of M on input x. We deterministically construct the configuration graph and run BFS/DFS from C_{start}, accepting iff C_{acc} is reachable.

Correctness. We show that the algorithm accepts x iff $x \in L$:

- If the algorithm accepts, then BFS/DFS found a path from C_{start} to C_{acc}. Since each edge represents a legal transition of M, this path corresponds to an accepting computation branch of M. Thus, $x \in L$.
- If $x \in L$, then M has at least one accepting computation branch, which corresponds to a path from C_{start} to C_{acc}. Since BFS/DFS explores all reachable vertices, the algorithm finds this path and accepts.

Running Time. Since M uses $O(S(n))$ space, each configuration can be encoded using $O(S(n))$ bits, so the number of vertices is $|V| = 2^{O(S(n))}$. Since each vertex has constant outdegree, $|E| = O(|V|) = 2^{O(S(n))}$. Finally, constructing the graph takes $2^{O(S(n))}$ time, and BFS/DFS takes $O(|V|+|E|) = 2^{O(S(n))}$. Thus, the total running time is $2^{O(S(n))}$.

3.4 Open Questions

$$\text{EXP} \stackrel{?}{=} \text{PSPACE}$$

$$\text{P} \stackrel{?}{=} \text{PSPACE}$$

$$\text{P} \stackrel{?}{=} \text{L}$$

If P = PSPACE, then P = NP = CoNP, etc...